EXPERIMENT 16

Write the Python program to implement a Feed Forward Neural Network

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program to implement a feed forward neural network with one hidden layer, trained with the backpropagation algorithm, and to use it to learn the XOR function.

Algorithm

Step 1: Define the training data: the four input pairs of the XOR problem and their target outputs (0, 1, 1, 0).

Step 2: Fix the network structure: 2 input neurons, 4 hidden neurons and 1 output neuron. Initialise the weights with small random values and the biases with zero. Choose the learning rate and the number of epochs.

Step 3: Forward pass: hidden = sigmoid(X x W1 + b1), then output = sigmoid(hidden x W2 + b2), where sigmoid(x) = 1 / (1 + e^-x).

Step 4: Calculate the error = target - output and the loss as the mean of the squared errors.

Step 5: Backward pass (backpropagation): output delta = error x output x (1 - output); hidden delta = (output delta x W2 transpose) x hidden x (1 - hidden).

Step 6: Update the weights and biases: W2 += hidden transpose x output delta x learning rate, W1 += X transpose x hidden delta x learning rate, and the biases with the sum of the deltas x learning rate.

Step 7: Repeat steps 3 to 6 for all epochs, printing the loss at regular intervals. Finally run a forward pass and print the output for each input; an output greater than 0.5 is classified as 1, otherwise 0.

192511013

Program

```
import numpy as np

np.random.seed(1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):
    return a * (1 - a)              # a = sigmoid(x)

# Training data: XOR problem
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Network: 2 input neurons -> 4 hidden neurons -> 1 output neuron
n_input, n_hidden, n_output = 2, 4, 1
W1 = np.random.uniform(-1, 1, (n_input, n_hidden))
b1 = np.zeros((1, n_hidden))
W2 = np.random.uniform(-1, 1, (n_hidden, n_output))
b2 = np.zeros((1, n_output))

learning_rate = 0.5
epochs = 10000

def forward(inputs):
    hidden = sigmoid(inputs @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return hidden, output

for epoch in range(epochs + 1):
    # forward pass
    hidden, output = forward(X)
    error = y - output
    loss = np.mean(error ** 2)
    if epoch % 2000 == 0:
        print(f"Epoch {epoch:>5}  Loss = {loss:.6f}")
    # backward pass (backpropagation)
    d_output = error * sigmoid_derivative(output)
    d_hidden = (d_output @ W2.T) * sigmoid_derivative(hidden)
    # update weights and biases
    W2 += hidden.T @ d_output * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += X.T @ d_hidden * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

print("\nPredictions after training:")
print(f"{'Input':<10}{'Target':<8}{'Output':<10}{'Class'}")
_, predictions = forward(X)
for inp, target, out in zip(X, y, predictions):
    print(f"{str(inp.tolist()):<10}{target[0]:<8}{out[0]:<10.4f}{int(out[0] > 0.5)}")
```

192511013

Output

```
Epoch     0  Loss = 0.249726
Epoch  2000  Loss = 0.002766
Epoch  4000  Loss = 0.000892
Epoch  6000  Loss = 0.000516
Epoch  8000  Loss = 0.000359
Epoch 10000  Loss = 0.000275

Predictions after training:
Input     Target  Output    Class
[0, 0]    0       0.0145    0
[0, 1]    1       0.9813    1
[1, 0]    1       0.9882    1
[1, 1]    0       0.0200    0
```

192511013